

電腦網路 Computer Network

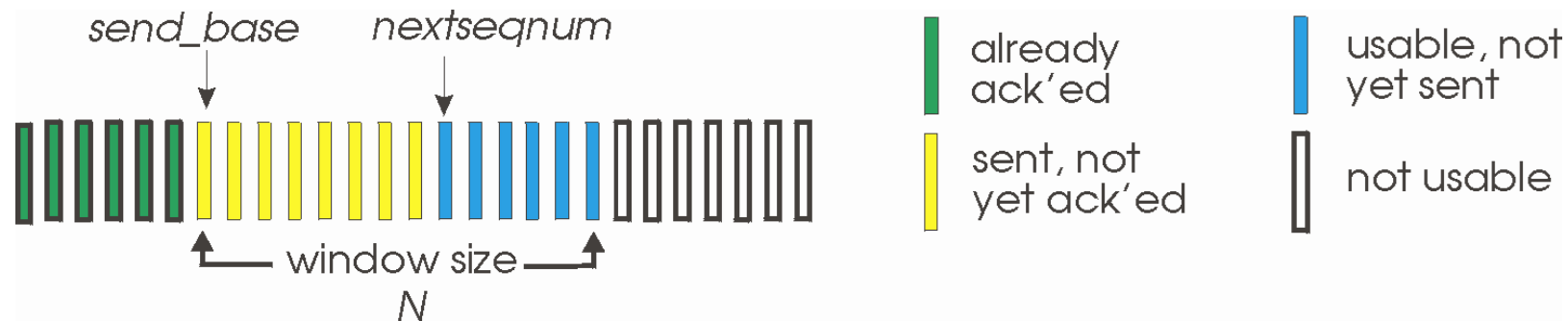
Homework 3

Department of Computer Science and Engineering,
National Sun Yat-sen University

Asst. Prof. ShengMin(Sam) Chiu

Homework 3

- Implementation of Reliable Data Transfer (Go-Back-N) over UDP and using Wireshark to analyze it.
- In this assignment, we will transition from using pre-built transport protocols to designing your own mechanism for reliability. Since UDP does not guarantee packet delivery, ordering, or integrity, your goal is to implement the Go-Back-N (GBN) protocol entirely within the application layer using Python. This task will give you hands-on experience managing a **sliding window**, handling **sequence numbers**, and implementing **timeout-driven retransmissions**.
- Unlike standard textbook GBN, which uses simple packet counts (0, 1, 2...), this assignment we will implement TCP-style Byte-based sequencing where sequence numbers track the byte offset of the payload data.



Protocol Specification & Header Design

- Header Packing and Parsing (using *struct* module)
 - Use Python's *struct* module to ensure correct byte ordering across different platforms.
- Every UDP datagram sent between the client and server must include a **fixed 8-byte header** packed using **Network Byte Order (like HW2)**.
- The structure is defined as follows:

Field Name	Length (Bytes)	Format Symbol	Description
Seq #	4	I	The identifying sequence number of the transmitted packet (used primarily by the sender).
Ack #	4	I	The cumulative acknowledgment number indicating the next expected packet sequence (used primarily by the receiver).

Implementation Constraints & Requirements

- Maximum Segment Size (MSS)
 - The Maximum Segment Size (**MSS**) is strictly capped at **1024 Bytes**.
 - MSS only measures the application data payload. When sending a packet, your actual UDP datagram size will be 1032 Bytes (8 bytes header + 1024 bytes data).
 - Packet Format: Total UDP Packet = Header (8bytes)+Data payload(up to 1024 bytes)
- There is a
 - ./client_dir/ directory in client-side.
 - ./server_dir/ directory in server-side.
- Setup: Download 10 images in eLearning platform (named test_0.jpg to test_9.jpg), and place them to ./server_dir/

Implementation Constraints & Requirements

- Client-side Functions
 - **<file_name>**: download the file “file_name” and save it to the ./client_dir/ directory.
- Server-side Functions
 - Serve files only from the ./server_dir/ folder.
 - Handle file transfers using binary mode (i.e. 'rb', 'wb').
 - The server only needs to serve one client at a time for this assignment.
- To trigger retransmissions on a local system, you must programmatically simulate random packet drops on the Receiver (Client) side.
 - **15% packet loss rate**
 - When a drop occurs, the client should completely bypass any processing or ACK generation for that segment.

Implementation Constraints & Requirements

- Personalized Configurations and Testing (Let X represent the last digit of your Student ID)
 - **Window Size (N):** Calculate as $N = (X \% 5) + 4$. Your sliding window size must fall strictly within the range of 4 to 8.
 - **Timeout Interval (T):** Calculate as $T = 0.5 + (X * 0.1)$ seconds. Your retransmission timer duration must fall strictly within the range of 0.5 to 1.4 seconds.
 - **Testing Case:** you MUST include getting the test_X.jpg
- To prevent confusion during implementation, please carefully note the distinction between the operational **units** used for the **sliding window** and the **sequence** tracking in this assignment:
 - Window Size (N) is measured in packets(segments): N strictly dictates the **maximum number of discrete packets** the sender is allowed to transmit without receiving an ACK.
 - Sequence & Acknowledgment Numbers are measured in Bytes: To mirror realistic TCP mechanics, the header fields **track the byte** offset of the data payload. Every full data segment advances the sequence progression by exactly 1024 bytes (our fixed MSS).

Wireshark Analysis & Report

- Use Wireshark to capture a complete testing case (which is downloading of the image test_X.jpg) and provide a detailed analytical report.
- **[CRITICAL] Your captured session MUST contain at least one simulated packet loss event.** If your luck is too good and you get 0 loss on the first try, you MUST re-run the programs until a loss occurs.
- Annotated Time Sequence Diagram: Draw a complete Time Sequence Diagram covering the entire testing case.
- Your diagram must include labels & descriptions:
 - tracking the packet sequence numbers (seq#) and acknowledgment numbers (ack#) **in the defined header**
 - Locating the simulated packet loss event in your traffic capture. Highlight the exact packet dropped by the client, document the subsequent out-of-order ACKs, and explicitly point out the subsequent GBN timeout retransmission burst.
 - Demonstrate how the time gap between a lost packet and its retransmission matches your personalized timeout calculation(T), and verify that your retransmission burst size aligns with your window size(N).

Submission

- Please **STRICTLY** follow the naming conventions and formatting requirements below for your submission
- Source Code:
 - Server-side: [ID]_hw3_server.py
 - Client-side: [ID]_hw3_client.py
- Report: Organize your report in **slide format** and **export it as a PDF file**.
 - File name: [ID]_hw3.pdf
- Final Package: Compress the three files above into a single ZIP archive.
 - File name: [ID]_hw3.zip
- Note: Replace [ID] with your actual student ID (e.g., B12345678).
- Deadline: 2 weeks

- Wrong formatting

> Frame 66: Packet, 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device\NPF_{Loopback}

> Null/Loopback

> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

> User Datagram Protocol, Src Port: 60262, Dst Port: 12000

> Logical Link Control

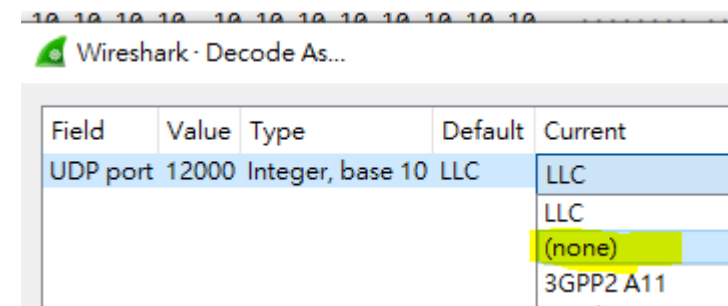
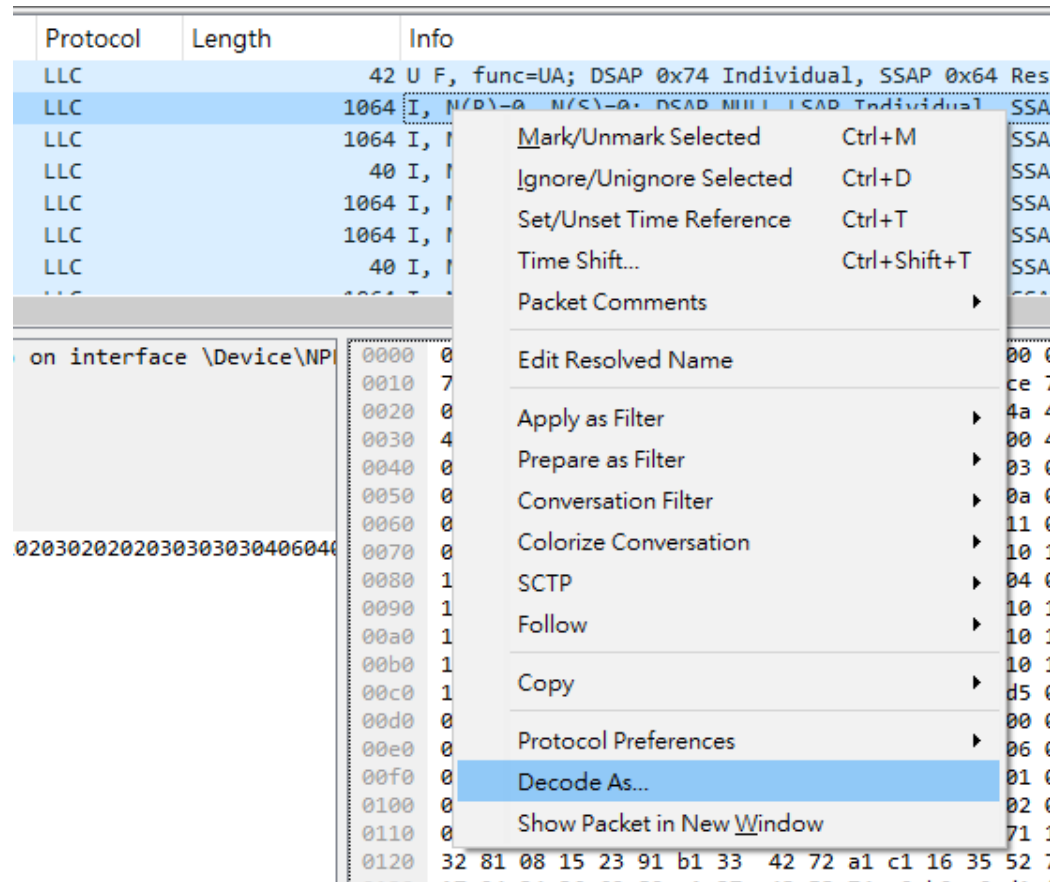
0000	02 00 00 00 45 00 00 26	9d c5 00 00 80 11 00 00	E..&.....
0010	7f 00 00 01 7f 00 00 01	eb 66 2e e0 00 12 01 9c	f.....
0020	74 65 73 74 5f 39 2e 6a	70 67	test 9.j p

Packet Format: Total UDP Packet = Header (8bytes)+Data payload(up to 1024 bytes)

Frame 67: Packet, 1064 bytes on wire (8512 bits), 1064 bytes captured (8512 bits) on interface \Device\NPF... Null/Loopback Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1 User Datagram Protocol, Src Port: 12000, Dst Port: 60262 Logical-Link Control Data (1028 bytes)		<pre> 0000 02 00 00 00 45 00 04 24 9d c6 00 00 80 11 00 00 E.\$..... 0010 7f 00 00 01 7f 00 00 01 2e e0 eb 66 04 10 ce 7c f... 0020 00 00 00 00 00 00 00 00 ff d8 ff e0 10 4a 46 J..... 0030 49 46 00 01 01 00 00 01 00 01 00 00 ff db 00 43 IF.....C... 0040 00 03 02 02 02 02 02 03 02 02 02 03 03 03 03 04 0050 06 04 04 04 04 04 08 06 06 05 06 09 08 0a 0a 09 0060 08 09 09 0a 0c 0f 0c 0a 0b 0e 0b 09 09 0d 11 0d 0070 0e 0f 10 10 11 10 0a 0c 12 13 12 10 13 0f 10 10 0080 10 ff db 00 43 01 03 03 03 04 03 04 08 04 04 08 C..... </pre>	
--	--	---	--

Hint

- Wrong formatting



No.	Time	Source	Destination	Protocol	Length	Info
66	15.841483500	127.0.0.1	127.0.0.1	UDP	42	60262 → 12000 Len=10
67	15.850998500	127.0.0.1	127.0.0.1	UDP	1064	12000 → 60262 Len=1032
68	15.851623400	127.0.0.1	127.0.0.1	UDP	1064	12000 → 60262 Len=1032
69	15.852273300	127.0.0.1	127.0.0.1	UDP	40	60262 → 12000 Len=8
70	15.852377900	127.0.0.1	127.0.0.1	UDP	1064	12000 → 60262 Len=1032
71	15.853379400	127.0.0.1	127.0.0.1	UDP	1064	12000 → 60262 Len=1032
72	15.854397600	127.0.0.1	127.0.0.1	UDP	40	60262 → 12000 Len=8
73	15.854408300	127.0.0.1	127.0.0.1	UDP	1064	12000 → 60262 Len=1032

```
> Frame 66: Packet, 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device\NPF_{Loopback}
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> User Datagram Protocol, Src Port: 60262, Dst Port: 12000
▼ Data (10 bytes)
  Data: 746573745f392e6a7067 [Length: 10]
```

Packet Format: Total UDP Packet = Header (8bytes)+Data payload(up to 1024 bytes)

Packet Format: Total UDP Packet = Header (8bytes)+Data payload(up to 1024 bytes)

No.	Time	Source	Destination	Protocol	Length	Info
66	15.841483500	127.0.0.1	127.0.0.1	UDP	42	60262 → 12000 Len=10
67	15.850998500	127.0.0.1	127.0.0.1	UDP	1064	12000 → 60262 Len=1032
68	15.851623400	127.0.0.1	127.0.0.1	UDP	1064	12000 → 60262 Len=1032
69	15.852273300	127.0.0.1	127.0.0.1	UDP	40	60262 → 12000 Len=8
70	15.852377900	127.0.0.1	127.0.0.1	UDP	1064	12000 → 60262 Len=1032
71	15.853379400	127.0.0.1	127.0.0.1	UDP	1064	12000 → 60262 Len=1032
72	15.854397600	127.0.0.1	127.0.0.1	UDP	40	60262 → 12000 Len=8
73	15.854403300	127.0.0.1	127.0.0.1	UDP	1064	12000 → 60262 Len=1032

[illegible]